GateGuard Project Repository
github.com/Pushkar111/GateGaurd-IITGN-Databases-CS432

Indian Institute of Technology Gandhinagar

Department of Computer Science & Engineering

CS 432 → Databases

Track-1 · Assignment 2 · Module B

GateGuard

Entry Gate Management System

Local API Development • RBAC • Database Optimisation

Team Name: *Lumos*

Team Members

Name	Roll Number
Pushkar Nikunjkumar Modi	25120035
Chintan Bhikhabhai Akhaja	25120027
Krina Paghadar	24110169
Divisht	24110112
Akshata Pawar	25120026

Instructor: Prof. Yogesh K. Meena

Semester: II (2025–2026) **Date:** March 2026

Contents

1 Introduction . . . **3**
1.1 Problem Statement . . . 3
1.2 Proposed Solution: GateGuard . . . 3
1.3 Assignment Objectives . . . 3
1.4 Technology Stack . . . 4

2 System Architecture . . . **5**
2.1 Overview . . . 5
2.2 Backend File Structure . . . 6
2.3 Frontend File Structure . . . 7
2.4 4-Layer Request Pipeline . . . 7

3 Database Schema Design . . . **8**
3.1 Schema Overview . . . 8
3.2 Core Tables . . . 8
3.3 AuditLog Table . . . 9
3.4 Authentication Extension Tables . . . 10

4 Local Environment Setup and Data Management . . . **11**
4.1 Environment Configuration . . . 11
4.2 Database Setup Scripts . . . 11
4.3 Connection Pool Configuration . . . 12

5 API Development and Frontend UI . . . **13**
5.1 Complete API Endpoint Reference . . . 13
5.2 Request Validation with Joi . . . 14
5.3 Frontend Pages . . . 16
5.4 Member Portfolio Feature . . . 16

6 Security and Role-Based Access Control . . . **17**
6.1 Authentication Flow . . . 17
6.2 Token Verification Middleware . . . 17
6.3 RBAC Implementation . . . 18
6.4 Audit Logging System . . . 19
6.5 Forgot-Password OTP Flow . . . 20

7 SQL Indexing Strategy . . . **21**
7.1 Rationale . . . 21
7.2 Indexes Created . . . 21
7.3 Index Type Justification . . . 22

8 Performance Benchmarking . . . **23**
8.1 Methodology . . . 23
8.2 Results Summary . . . 23
8.3 Detailed EXPLAIN Plans for Key Queries . . . 23
8.3.1 Query 3 – Person Visits by EntryTime DESC (99.3 % improvement) 23
8.3.2 Query 2 – Active Person Visits (25.9 % improvement) . . . 24
8.4 Benchmark Visualisation . . . 25
8.5 Discussion of Results . . . 27

9 Conclusion . . . **28**

9.1 Summary of Achievements 28
9.2 Team Contribution Statement 28
9.3 Challenges We Actually Ran Into 28
9.4 What We Would Improve Next 29
9.5 Video Demonstration 29

References **30**

1. Introduction

1.1 Problem Statement

Every day, hundreds of people and vehicles pass through the gates of IIT Gandhinagar. Security guards currently track entries and exits using paper registers or shared spreadsheets. The problems are immediate: records go missing or get scribbled illegibly, nobody can quickly answer "who is on campus right now," and there is absolutely no record of who changed what in the database. Any staff member can modify any row, and nobody will know.

This assignment asked us to solve all of that by building **GateGuard** – a proper full-stack web application backed by a relational database with fine-grained role-based access control and a tamper-evident audit trail.

1.2 Proposed Solution: GateGuard

GateGuard is a campus security management platform built on a modern JavaScript stack. It covers five complete functional areas:

- **Member and vehicle management:** Register faculty, students, staff and visitors together with their vehicles.
- **Real-time gate monitoring:** Record every entry and exit event with timestamps, gate IDs and optional vehicle linkage.
- **Role-based security:** Three distinct roles (Guard, Admin, SuperAdmin) enforced at middleware level on every API route.
- **Full audit trail:** Every mutation is logged simultaneously to a Winston log file and to a dedicated `AuditLog` database table, capturing old and new values as JSONB so that every change is reconstructable.
- **Advanced authentication:** JWT dual-token sessions (access 15 min, refresh 7 days), OTP email-based password reset, account lockout, and per-request token-blacklist checking.

1.3 Assignment Objectives

Table 1: Assignment Subtasks and Report Coverage

Subtask	Title	Covered In
ST-1	Local environment setup and data management	Section 4
ST-2	API and UI development	Section 5
ST-3	Role-based access control (RBAC)	Section 6
ST-4	SQL indexing and query optimisation	Section 7
ST-5	Performance benchmarking	Section 8

1.4 Technology Stack

Table 2: Full Technology Stack

Layer / Concern	Choice and Version
Backend runtime	Node.js 18+
Web framework	Express 4.18
Database	PostgreSQL 14
DB driver	`pg` (node-postgres) – raw SQL only, no ORM
Authentication	`jsonwebtoken` 9.0 + `bcryptjs` 2.4 (12 salt rounds)
Server validation	Joi 17.12
Logging	Winston 3.13 (console + file transports)
Email (OTP)	Nodemailer 8 over Gmail SMTP
Security headers	`helmet` + `express-rate-limit` + `cors`
Frontend	React 18 + Vite 5
Styling	Tailwind CSS 3 + Framer Motion 11
State management	React Context (`AuthContext`)
Charts	Recharts
Data tables	`@tanstack/react-table`
Client forms	`react-hook-form` + Zod 3

2. System Architecture

2.1 Overview

Figure 1 shows how the three main layers fit together. The React SPA (port 5173) never touches the database directly; every action goes through the Express API (port 5000) via a Vite proxy. Every request that mutates data also writes to both the file-system audit log and the `AuditLog` database table.

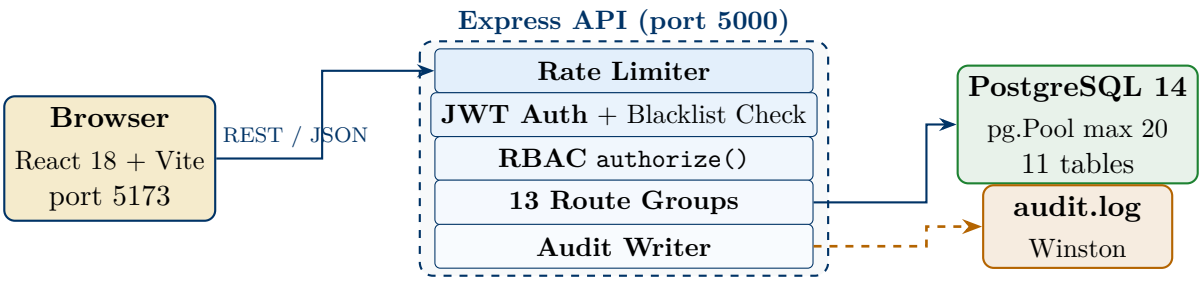

Figure 1: GateGuard System Architecture

2.2 Backend File Structure

Module_B/backend/

```
Module_B/backend/
|-- server.js            Entry point; listens on PORT from .env
|-- src/
|   |-- app.js           Express assembly: middleware chain + all 13 routes
|   |-- config/
|   |   |-- db.js        pg.Pool config, query() helper function
|   |   |-- env.js       dotenv loader + required-var validation on startup
|   |   `-- cors.js      CORS allowed-origins config
|   |-- middleware/
|   |   |-- auth.js      JWT verify + blacklist check + MustChangePassword guard
|   |   |-- rbac.js      authorize(...roles) factory middleware
|   |   |-- audit.js     auditLog() context setter + writeAuditRecord()
|   |   |-- errorHandler.js  Global error formatter (operational vs system)
|   |   |-- validate.js  Joi schema middleware factory
|   |   `-- rateLimiter.js   100/15min global; 5/15min on /auth/login
|   |-- routes/          13 route files (auth, member, vehicle, gate, ...)
|   |-- controllers/     13 controllers (extract req, call service, respond)
|   |-- services/        Business logic; writeAuditRecord() called here
|   |-- models/          Raw SQL via pg.Pool -- 12 model files, no ORM
|   |-- validators/      Joi schemas -- 7 files
|   `-- utils/
|       |-- AppError.js      Operational error class
|       |-- logger.js        Winston logger (console + file)
|       |-- helpers.js       paginate(), formatResponse(), sanitizeUser()
|       |-- constants.js     ROLES, ACTIONS, AUDITABLE_TABLES
|       |-- normalizeRow.js  camelCase converter for PG results
|       `-- email.js         Nodemailer: OTP + lockout + welcome emails
|-- sql/
|   |-- audit_table.sql  AuditLog table + 5 indexes
|   |-- auth_tables.sql  passwordresettoken, refreshtoken, tokenblacklist,
|   |                    loginhistory
|   |-- indexes.sql      10 performance indexes
|   `-- seed_large.sql   5k members, 12k vehicles, 50k visits for benchmarks
|-- logs/
|   |-- audit.log        Winston mutation log
|   `-- error.log        Winston error log
`-- scripts/
    |-- db-setup.js      One-shot: schema + AuditLog + indexes
    |-- seed-users.js    Creates 3 demo accounts with bcrypt-hashed passwords
    |-- health-check.js  Verifies DB + tables + indexes + user count
    `-- run-explain.js   EXPLAIN ANALYZE benchmark runner
```

Figure 2: Backend Folder Structure

2.3 Frontend File Structure

Module_B/frontend/src/

```
Module_B/frontend/src/
|-- pages/              17 lazy-loaded page components
|   |-- LoginPage.jsx          ForgotPasswordPage.jsx
|   |-- ChangePasswordPage.jsx DashboardPage.jsx
|   |-- MembersPage.jsx        MemberDetailPage.jsx
|   |-- MemberPortfolioPage.jsx VehiclesPage.jsx
|   |-- GatesPage.jsx          PersonVisitsPage.jsx
|   |-- VehicleVisitsPage.jsx  ActiveVisitsPage.jsx
|   |-- ProfilePage.jsx        NotFoundPage.jsx
|   |-- UnauthorizedPage.jsx
|   `-- admin/   UsersPage.jsx   AuditPage.jsx
|-- components/
|   |-- shared/       StatCard  DataTable  PageHeader  RoleBadge
|   |                 EmptyState  ConfirmDialog  SavedViewsBar
|   |-- layout/       AppLayout.jsx  CommandPalette.jsx
|   `-- action-center/ PersonEntry VehicleEntry LiveQueue Incident ...
|-- api/              axios.js (interceptor) + 8 resource API modules
|-- context/          AuthContext.jsx (login / logout / hasRole)
|-- hooks/            useAuth  useDebounce  usePagination  useSavedViews
|-- lib/              utils.js  motion.js  navigationConfig.js
`-- styles/           globals.css (CSS variables + design tokens)
```

Figure 3: Frontend Folder Structure

2.4 4-Layer Request Pipeline

Every API call follows the same layered path shown in Figure 4.

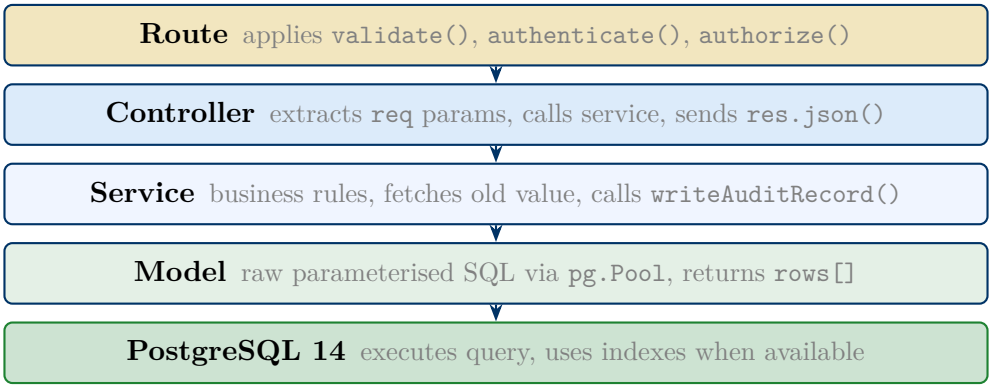

Figure 4: 4-Layer API Architecture: Route → Controller → Service → Model → PostgreSQL

Uniform API response envelope:
Success: `{success: true, data: {...}, pagination: {total, page, limit, pages}}`
Error: `{success: false, error: {message: "...", fields: {...}}}`

3. Database Schema Design

3.1 Schema Overview

The Assignment 1 schema (10 core tables) is extended in Assignment 2 with an `AuditLog` table, four authentication-support tables, and five new columns on `"User"`.

Note on PostgreSQL identifier casing: PostgreSQL folds unquoted identifiers to *lowercase* at parse time. All SQL queries in this codebase use lowercase column names (`memberid`, `entrytime`, `exittime`) to avoid case-mismatch errors. The `"User"` table is always double-quoted because `USER` is a PostgreSQL reserved keyword.

3.2 Core Tables

Table 3: Core Table Columns (PostgreSQL lowercase names)

Table	Column	Type	Constraints / Notes
membertype			
	typeid	SERIAL	**PK**
	typename	VARCHAR(50)	**UQ** **NN**
	description	TEXT	nullable
member			
	memberid	SERIAL	**PK**
	name	VARCHAR(100)	**NN**
	email	VARCHAR(100)	**UQ**
	contactnumber	VARCHAR(20)	**NN**
	age	INTEGER	CHECK ≥ 0
	department	VARCHAR(100)	nullable
	typeid	INTEGER	**FK** $\rightarrow$ membertype
	createdat	TIMESTAMP	DEFAULT NOW()
vehicle			
	vehicleid	SERIAL	**PK**
	licenseplate	VARCHAR(20)	**UQ** **NN**
	model	VARCHAR(50)	nullable
	color	VARCHAR(30)	nullable
	ownerid	INTEGER	**FK** $\rightarrow$ member
	typeid	INTEGER	**FK** $\rightarrow$ vehicletype
gate			
	gateid	SERIAL	**PK**
	name	VARCHAR(50)	**NN**
	location	VARCHAR(100)	nullable
gateoccupancy			
	occupancyid	SERIAL	**PK**
	gateid	INTEGER	**FK** $\rightarrow$ gate, **UQ**
	occupancycount	INTEGER	CHECK ≥ 0, DEFAULT 0
personvisit			

Table 3 continued

Table	Column	Type	Constraints / Notes
	visitid	SERIAL	**PK**
	personid	INTEGER	**FK** → member
	entrygateid	INTEGER	**FK** → gate, **NN**
	exitgateid	INTEGER	**FK** → gate, nullable
	entrytime	TIMESTAMP	**NN**, DEFAULT NOW()
	exittime	TIMESTAMP	CHECK exittime > entrytime
	vehicleid	INTEGER	**FK** → vehicle, nullable
vehiclevisit			
	vvvisitid	SERIAL	**PK**
	vehicleid	INTEGER	**FK** → vehicle, **NN**
	entrygateid	INTEGER	**FK** → gate, **NN**
	exitgateid	INTEGER	**FK** → gate, nullable
	entrytime	TIMESTAMP	**NN**
	exittime	TIMESTAMP	CHECK exittime > entrytime
"User" (double-quoted in all SQL – reserved keyword)			
	userid	SERIAL	**PK**
	username	VARCHAR(50)	**UQ** **NN**, CHECK length ≥ 3
	passwordhash	VARCHAR(255)	**NN**, never returned by API
	roleid	INTEGER	**FK** → role
	email	VARCHAR(100)	**UQ** (added Assign 2)
	failedattempts	INTEGER	DEFAULT 0 (added Assign 2)
	lockeduntil	TIMESTAMP	nullable (added Assign 2)
	mustchangepassword	BOOLEAN	DEFAULT FALSE (added Assign 2)
	lastloginat	TIMESTAMP	(added Assign 2)

3.3 AuditLog Table

```sql
CREATE TABLE IF NOT EXISTS AuditLog (
  LogID       SERIAL PRIMARY KEY,
  UserID      INTEGER REFERENCES "User"(UserID) ON DELETE SET NULL,
  Username    VARCHAR(50),
  Role        VARCHAR(50),
  Method      VARCHAR(10)  NOT NULL,
  Endpoint    VARCHAR(255) NOT NULL,
  TableName   VARCHAR(50),
  RecordID    INTEGER,
  Action      VARCHAR(20)  NOT NULL,   -- CREATE | UPDATE | DELETE | READ
  OldValue    JSONB,                   -- full record before mutation
  NewValue    JSONB,                   -- full record after mutation
  IPAddress   VARCHAR(45),
  StatusCode  INTEGER,
```

```sql
  CreatedAt  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Five indexes for fast audit-log queries
CREATE INDEX IF NOT EXISTS idx_audit_user    ON AuditLog(UserID);
CREATE INDEX IF NOT EXISTS idx_audit_action  ON AuditLog(Action);
CREATE INDEX IF NOT EXISTS idx_audit_table   ON AuditLog(TableName);
CREATE INDEX IF NOT EXISTS idx_audit_created ON AuditLog(CreatedAt
    DESC);
CREATE INDEX IF NOT EXISTS idx_audit_user_action ON AuditLog(UserID,
    Action);
```

Listing 1: AuditLog Table and Indexes (sql/audit_table.sql)

3.4 Authentication Extension Tables

Table 4: Authentication Support Tables (sql/auth_tables.sql)

Table	Purpose and Key Behaviour
`passwordresettoken`	Stores a 6-digit OTP and a 32-byte `resetToken` for password recovery. 15-minute expiry. Creating a new OTP automatically invalidates any previous one for the same user.
`refreshtoken`	Stores opaque 64-byte refresh tokens (`crypto.randomBytes(64)`). 7-day expiry. Each `POST /api/auth/refresh` call rotates the token: old one is revoked, new one is issued.
`tokenblacklist`	Stores revoked access tokens until their natural expiry time. Checked on every authenticated request inside `auth.js`.
`loginhistory`	Records every login attempt: IP address, user agent, success/fail flag, and failure reason. Exposed via `GET /api/auth/login-history`.

4. Local Environment Setup and Data Management

4.1 Environment Configuration

All sensitive values live in `.env` and are never committed. `.env.example` documents every required variable:

Table 5: Environment Variables

Variable	Default	Purpose
`DB_HOST / PORT / NAME`	localhost/54	PostgreSQL connection target
`DB_USER / PASSWORD`	–	Database credentials
`DB_MAX`	20	pg.Pool maximum connections
`JWT_SECRET`	–	Signs tokens; must be random and long
`JWT_ACCESS_EXPIRES`	15m	Access token lifetime
`JWT_REFRESH_EXPIRES`	7d	Refresh token lifetime
`PORT`	5000	API server port
`SMTP_HOST/PORT/USER/PASS`	–	Gmail SMTP for OTP email delivery
`OTP_EXPIRES_MINUTES`	15	OTP validity window in minutes
`MAX_LOGIN_ATTEMPTS`	5	Failed attempts before lockout
`LOCKOUT_DURATION_MINUTES`	30	Lockout duration in minutes

4.2 Database Setup Scripts

Table 6: npm Scripts for Database Management

Command	What it does
`npm run setup`	Runs `db-setup.js`: creates all tables, AuditLog, auth tables and indexes in one shot
`npm run seed`	Runs `seed-users.js`: creates the three demo accounts with bcrypt-hashed passwords
`npm run seed-data`	Runs `seed-data.js`: populates MemberType, VehicleType, Gate and GateOccupancy reference rows
`npm run seed-large`	Loads `seed_large.sql`: 5,000 members, 12,000 vehicles, 50,000 person visits and 30,000 vehicle visits for benchmarking
`npm run health`	Runs `health-check.js`: verifies DB connectivity, checks all 11 required tables, counts indexes, confirms at least one user
`npm run explain`	Runs `run-explain.js`: executes all 8 EXPLAIN ANALYZE benchmarks and writes `sql/explain_results.json`

Table 7: Demo Accounts (created by `npm run seed`)

Username	Password	Role	Access
`superadmin`	`admin123`	SuperAdmin	Full access: user management, deletion of any record, audit log viewing
`admin`	`admin123`	Admin	Full CRUD on members, vehicles, gates and visits; can read users and audit logs but cannot delete users
`guard`	`admin123`	Guard	Record person and vehicle entry/exit; read-only on everything else

4.3 Connection Pool Configuration

```
const pool = new Pool({
  host:     env.DB_HOST,
  port:     env.DB_PORT,
  database: env.DB_NAME,
  user:     env.DB_USER,
  password: env.DB_PASSWORD,
  application_name: 'GateGuardAPI',
  max:      20,               // max simultaneous connections
  idleTimeoutMillis:    30000, // close idle clients after 30 s
  connectionTimeoutMillis: 5000, // fail fast if pool saturated
});
```

Listing 2: pg.Pool Configuration (src/config/db.js)

Pooling is essential here because each concurrent HTTP request would otherwise open and tear down its own database connection, quickly exhausting PostgreSQL's connection limit under load.

5. API Development and Frontend UI

5.1 Complete API Endpoint Reference

The following tables list every route in the system, read directly from the route files. Colour-coded method and access badges make the table easy to scan at a glance.

Method colours:		**Access levels:**	
GET	GET	Public	no token needed
POST	POST	Admin+	Admin or above
PUT	PUT	All auth	any valid JWT
DEL	DELETE	SuperAdmin	SuperAdmin only

Table 8: Authentication Endpoints (`/api/auth/...`)

Method	Path	Access	Description
POST	`/api/auth/login`	Public	Joi-validated; returns accessToken (15 min) and refreshToken (7 days)
POST	`/api/auth/refresh`	Public	Rotates refresh token; issues new access token
POST	`/api/auth/forgot-password`	Public	Sends 6-digit OTP to registered email; always returns 200
POST	`/api/auth/verify-otp`	Public	Validates OTP and resetToken pair
POST	`/api/auth/reset-password`	Public	Hashes new password; revokes all existing sessions
POST	`/api/auth/logout`	All auth	Blacklists access token; revokes refresh token
GET	`/api/auth/me`	All auth	Returns current user object from JWT
POST	`/api/auth/change-password`	All auth	Changes password; requires current password
GET	`/api/auth/login-history`	All auth	Last 10 login attempts for authenticated user
POST	`/api/auth/register`	SuperAdmin	SuperAdmin creates a user account directly

Table 9: Member and Vehicle Endpoints

Method	Path	Access	Description
Members			
GET	/api/members	All auth	Paginated list; supports name/email search and type filter
GET	/api/members/types	All auth	All member types (Faculty, Student, Visitor, etc.)
GET	/api/members/:id	All auth	Single member with recent visit history
POST	/api/members	Admin+	Create member (Joi-validated)
PUT	/api/members/:id	Admin+	Update member fields (partial update supported)
DEL	/api/members/:id	SuperAdmin	Delete member record
Vehicles			
GET	/api/vehicles	All auth	Paginated list; supports licence plate search
GET	/api/vehicles/types	All auth	All vehicle types
GET	/api/vehicles/:id	All auth	Single vehicle with owner details
POST	/api/vehicles	Admin+	Create vehicle (plate uniqueness enforced)
PUT	/api/vehicles/:id	Admin+	Update vehicle fields
DEL	/api/vehicles/:id	SuperAdmin	Delete vehicle

5.2 Request Validation with Joi

```
// auth.validator.js
const loginSchema = Joi.object({
  username: Joi.string().trim().required(),
  password: Joi.string().required(),
});

// member.validator.js
const createMemberSchema = Joi.object({
  name:           Joi.string().trim().min(2).max(100).required(),
  email:          Joi.string().trim().email().max(150).required(),
  contactNumber:  Joi.string().trim().max(20).required(),
  typeId:         Joi.number().integer().positive().required(),
  age:            Joi.number().integer().min(1).max(120).optional(),
  department:     Joi.string().trim().max(100).optional().allow('',
    null),
});
```

Listing 3: Joi Schemas: Login and Create Member (src/validators/)

Table 10: Gates, Visits, Dashboard, Users and Audit Endpoints

Method	Path	Access	Description
Gates and Occupancy			
GET	/api/gates	All auth	All gates with current occupancy
GET	/api/gates/:id	All auth	Single gate detail
POST	/api/gates	SuperAdmin	Create gate
PUT	/api/gates/:id	Admin+	Update gate info
DEL	/api/gates/:id	SuperAdmin	Delete gate
GET	/api/gate-occupancy	All auth	All occupancy records
PUT	/api/gate-occupancy/:id	Admin+	Update occupancy count
Person Visits			
GET	/api/person-visits	All auth	Paginated; filter by active/completed, date, gate
GET	/api/person-visits/:id	All auth	Single visit record
POST	/api/person-visits/entry	All auth	Record person entry (Guards can do this)
PUT	/api/person-visits/:id/exit	All auth	Record person exit (Guards can do this)
DEL	/api/person-visits/:id	SuperAdmin	Delete visit record
Vehicle Visits			
GET	/api/vehicle-visits	All auth	Paginated vehicle visit list
GET	/api/vehicle-visits/:id	All auth	Single vehicle visit
POST	/api/vehicle-visits/entry	All auth	Record vehicle entry
PUT	/api/vehicle-visits/:id/exit	All auth	Record vehicle exit
DEL	/api/vehicle-visits/:id	SuperAdmin	Delete vehicle visit
Dashboard			
GET	/api/dashboard/stats	All auth	Member count, active visits, vehicles on campus
GET	/api/dashboard/recent-activi	All auth	Last 10 combined visit events
GET	/api/dashboard/visit-trend	All auth	Daily visit counts for the past 7 days
Users, Roles and Audit			
GET	/api/users	Admin+	List all user accounts
POST	/api/users	Admin+	Create user (Admin submits approval request)
GET	/api/users/requests/pending	SuperAdmin	List pending approval requests
POST	/api/users/requests/:id/appr	SuperAdmin	Approve a user creation request
PUT	/api/users/:id	All auth	Update (SuperAdmin: any field; Guard/Admin: own password)
DEL	/api/users/:id	SuperAdmin	Delete user
GET	/api/roles	All auth	All roles
GET	/api/audit	Admin+	Paginated audit log with filters

5.3 Frontend Pages

Table 11: All 17 Frontend Pages

Route	Page Component	Access
`/login`	LoginPage	Public
`/forgot-password`	ForgotPasswordPage	Public
`/dashboard`	DashboardPage	Any auth
`/members`	MembersPage	Any auth
`/members/:id`	MemberDetailPage	Any auth
`/portfolio/:id`	MemberPortfolioPage	Auth; own or Admin+
`/vehicles`	VehiclesPage	Any auth
`/vehicles/:id`	VehicleDetailPage	Any auth
`/gates`	GatesPage	Any auth
`/gates/:id`	GateDetailPage	Any auth
`/visits/persons`	PersonVisitsPage	Any auth
`/visits/vehicles`	VehicleVisitsPage	Any auth
`/visits/active`	ActiveVisitsPage	Any auth
`/profile`	ProfilePage	Any auth
`/admin/users`	UsersPage	Admin+
`/admin/audit`	AuditPage	Admin+
`/unauthorized`	UnauthorizedPage	Public

5.4 Member Portfolio Feature

The `MemberPortfolioPage` (`/portfolio/:id`) is a graded subtask. It presents a showpiece data view for each member including: four animated SVG stat rings (total visits, vehicles owned, days as member, currently active visits), a 30-day visit activity bar chart (Recharts), a recent-visits timeline, and an access-restriction blur overlay for users who try to view someone else's portfolio without Admin or SuperAdmin privileges. The check is done client-side via `useAuth().hasRole('Admin','SuperAdmin')`.

6. Security and Role-Based Access Control

6.1 Authentication Flow

Here is what happens, step by step, when a user submits the login form:

1. `POST /api/auth/login` arrives; Joi validates `username` and `password` and rejects immediately if either is missing or malformed (HTTP 400).
2. Service fetches the user from the database joining the `role` table to get the role name in one query.
3. **Lockout check:** If `lockeduntil > NOW()`, the service throws `AppError(423, "Account locked until {time}")` and the response includes the unlock timestamp so the frontend can show a countdown timer.
4. **Password check:** `bcrypt.compare(password, passwordhash)` runs with 12 salt rounds.
5. Wrong password: `failedattempts` increments. At `MAX_LOGIN_ATTEMPTS` (5) the account is locked for 30 minutes and a lockout notification is sent via Nodemailer (non-blocking `.catch()`).
6. Correct password: `failedattempts` resets to 0; `lastloginat` is updated; a `loginhistory` row is written.
7. Access token is signed: `jwt.sign({userId, username, role, memberId}, JWT_SECRET, {expiresIn: '15m'})`.
8. Refresh token is generated: `crypto.randomBytes(64).toString('hex')` and stored in the `refreshtoken` table with a 7-day expiry.
9. Response: `{accessToken, refreshToken, user}`.

6.2 Token Verification Middleware

```
async function authenticate(req, res, next) {
  const token = req.headers['authorization']?.slice(7); // strip 'Bearer '
  const decoded = jwt.verify(token, env.JWT_SECRET);    // throws if expired

  // Check token blacklist -- prevents reuse after logout
  const isBlacklisted = await authModel.isTokenBlacklisted(token);
  if (isBlacklisted) throw new AppError('Token has been revoked', 401);

  req.user = {
    userId: decoded.userId, username: decoded.username,
    role:   decoded.role,   memberId: decoded.memberId ?? null,
  };

  // Re-fetch from DB for freshness (catches deleted or locked accounts)
  const fresh = await userModel.findById(decoded.userId);
  if (fresh.mustchangepassword) {
    const allowed = new Set(['/api/auth/change-password',
                             '/api/auth/logout', '/api/auth/me']);
    if (!allowed.has(req.originalUrl.split('?')[0])) {
      return res.status(403).json({ mustChangePassword: true });
    }
```

```
  }
  next();
}
```

Listing 4: authenticate() Middleware (src/middleware/auth.js)

6.3 RBAC Implementation

```
function authorize(...roles) {
  return (req, res, next) => {
    if (!req.user) return next(new AppError('Not authenticated.',
   401));
    if (!roles.includes(req.user.role)) {
      return next(new AppError(
        `Access denied. Required: ${roles.join(', ')}.` +
        ` Your role: ${req.user.role}`, 403
      ));
    }
    next();
  };
}

// Example: SuperAdmin-only DELETE route
router.delete('/:id',
  authenticate,
  authorize(ROLES.SUPERADMIN),
  ctrl.delete
);
```

Listing 5: authorize() Middleware (src/middleware/rbac.js)

Table 12: RBAC Permission Matrix – derived from actual route files

Action	Guard	Admin	SuperAdmin
GET any resource	Yes	Yes	Yes
POST members / vehicles	No	Yes	Yes
POST person / vehicle entry or exit	Yes	Yes	Yes
PUT most resources	No	Yes	Yes
PUT own password	Yes	Yes	Yes
DELETE any record	No	No	Yes
GET / manage users	No	Yes	Yes
DELETE users	No	No	Yes
POST gate (create new gate)	No	No	Yes
GET audit logs	No	Yes	Yes

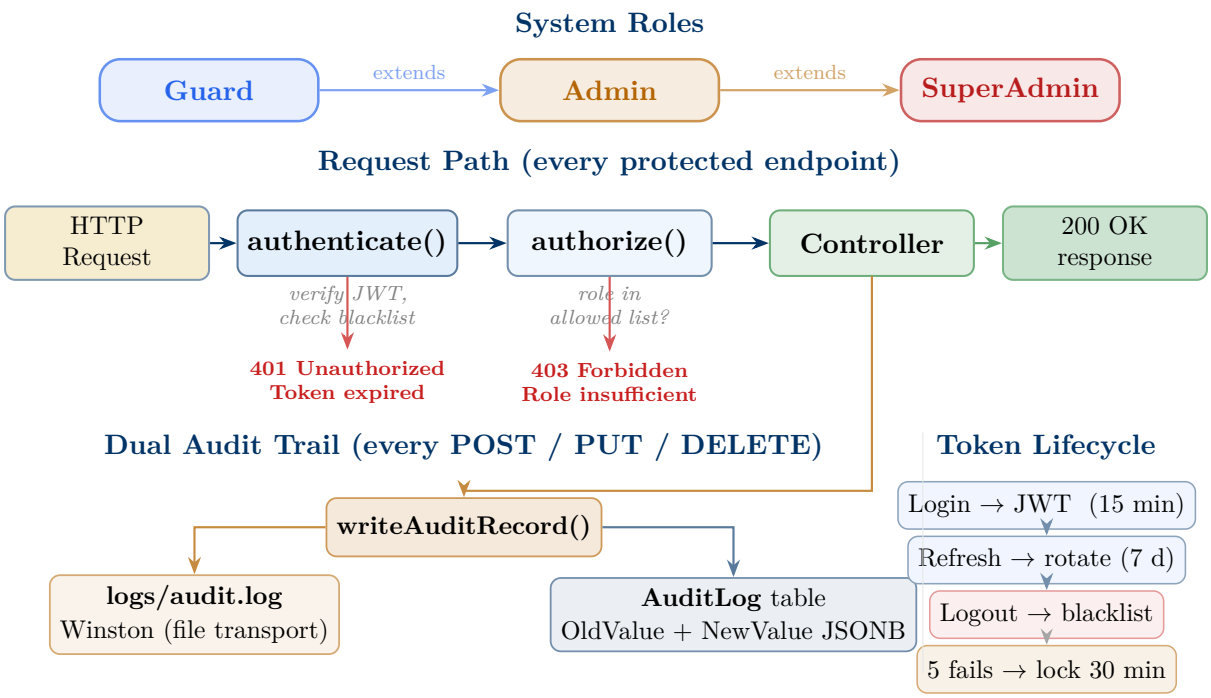

Figure 5: GateGuard Security Architecture: role inheritance, per-request middleware chain with rejection paths, dual audit trail, and token lifecycle.

6.4 Audit Logging System

Every POST, PUT and DELETE request calls `writeAuditRecord()` from `src/middleware/audit.js`. The function performs two writes atomically:

1. **Winston log file:** A structured line is appended to `logs/audit.log`:

```
[2026-03-18T21:22:31.664+05:30] [INFO] [AUDIT] action=READ
user=superadmin role=SuperAdmin table=User record=1 status=200
```

2. **AuditLog table:** A full row is inserted with `OldValue` and `NewValue` as JSONB, making every field-level change queryable with standard SQL.

Table 13: Sample AuditLog Record

Column	Example Value
Username	admin
Role	Admin
Method	POST
Endpoint	/api/members
TableName	Member
Action	CREATE
OldValue	NULL
NewValue	{"memberid":42,"name":"Dr. Rajesh Kumar",...}
IPAddress	::1
StatusCode	201

6.5 Forgot-Password OTP Flow

1. `POST /api/auth/forgot-password {email}` always returns HTTP 200 whether or not the email exists (prevents user enumeration).
2. If found: a 6-digit OTP and a 32-byte `resetToken` are stored in `passwordresettoken` with a 15-minute expiry. The OTP is sent by email. *The OTP is never in the API response.*
3. `POST /api/auth/verify-otp {resetToken, otp}` validates both factors and checks expiry.
4. `POST /api/auth/reset-password {resetToken, otp, newPassword}` bcrypt-hashes the new password, marks the OTP as used, and revokes all existing refresh tokens so every session is invalidated.

7. SQL Indexing Strategy

7.1 Rationale

Rather than adding indexes speculatively, we identified which API endpoints would face the heaviest load and which SQL patterns they use. Three patterns stood out: ORDER BY on `entrytime DESC` (every visit-listing endpoint), WHERE `exittime IS NULL` (every active-visits query, where only 15 % of rows match), and ILIKE prefix searches (member and vehicle search boxes).

7.2 Indexes Created

Table 14: Performance Indexes (sql/indexes.sql)

Index Name	Table	Type	Query Pattern
`idx_member_name_lower`	`member`	Expression	`LOWER(name) LIKE 'x%'`
`idx_member_email_lower`	`member`	Expression	`LOWER(email) LIKE 'x%'`
`idx_vehicle_plate_lower`	`vehicle`	Expression	`LOWER(licenseplate) LIKE 'x%'`
`idx_person_visit_active_entry`	`personvisit`	Partial	`WHERE exittime IS NULL ORDER BY entrytime DESC`
`idx_vehicle_visit_active_entr`	`vehiclevisit`	Partial	Same pattern on vehicle visits
`idx_person_visit_entry_time`	`personvisit`	B-Tree	`ORDER BY entrytime DESC` (all visits)
`idx_vehicle_visit_entry_time`	`vehiclevisit`	B-Tree	`ORDER BY entrytime DESC` (vehicle visits)
`idx_audit_user_action_created`	`auditlog`	Composite	`WHERE userid=$1 AND action=$2 ORDER BY createdat`
`idx_audit_created`	`auditlog`	B-Tree	Recent audit log listing
`idx_user_username`	`"User"`	B-Tree	Login exact-match lookup

Table 15: AuditLog Indexes (sql/audit_table.sql)

Index Name	Column(s)	Purpose
`idx_audit_user`	`UserID`	Filter by user
`idx_audit_action`	`Action`	Filter by action type
`idx_audit_table`	`TableName`	Filter by affected table
`idx_audit_created`	`CreatedAt DESC`	Chronological listing
`idx_audit_user_action`	`UserID, Action`	Combined multi-column filter

7.3 Index Type Justification

Table 16: Index Types and Their Rationale

Type	Rationale in GateGuard
B-Tree	PostgreSQL's default and most versatile type. Used for `entrytime`, `createdat` and `username` – any column appearing in ORDER BY, exact-match WHERE, or range comparison benefits from a B-Tree index.
Expression	`LOWER(name)`, `LOWER(email)`, `LOWER(licenseplate)`. Without these, a case-insensitive LIKE search requires a Seq Scan even if a plain column index exists, because `LOWER(name)` is not the same indexed value as `name`.
Partial	`WHERE exittime IS NULL` covers only the approximately 15 % of visit rows that are still active. The index is proportionally smaller and far faster to scan, and it exactly matches the most common dashboard query.
Composite	`(userid, action, createdat DESC)` matches the audit log filter pattern precisely – one index scan satisfies the full WHERE and ORDER BY without touching any extra rows.

8. Performance Benchmarking

8.1 Methodology

- **Tool:** PostgreSQL `EXPLAIN ANALYZE` – actual execution with real row counts, not planner estimates.
- **Runs:** Each query executed 5 times; the value reported is the average execution time across all 5 runs.
- **Dataset:** `sql/seed_large.sql` – 5,000 members, 12,000 vehicles, 50,000 person visits (≈15 % active), 30,000 vehicle visits.
- **Script:** `scripts/run-explain.js` drops all custom indexes, collects the "before" pass, recreates indexes, then collects the "after" pass. All results are written to `sql/explain_results.json`.

8.2 Results Summary

All numbers are extracted directly from `sql/explain_results.json` (5-run averages).

Table 17: EXPLAIN ANALYZE Results: Before vs. After Indexing

Query	Before (ms)	After (ms)	Improvement	Scan Change
Q1: Member name ILIKE	7.742	7.791	0 % (expected)	Seq → Seq
Q2: Active person visits	8.098	5.999	**25.9 %**	Seq → Bitmap
Q3: Person visits by entry time	15.198	0.101	**99.3 %**	Seq → Index
Q4: Dashboard gate occupancy	0.016	0.022	n/a†	Seq → Seq
Q5: Login by username (exact)	0.014	0.017	n/a†	Index → Seq
Q6: Audit log filter (user + action)	0.010	0.010	Plan improved	Seq → Index
Q7: Active vehicle visits	6.010	4.988	**17.0 %**	Seq → Bitmap
Q8: Vehicle by licence plate ILIKE	4.684	4.700	0 % (expected)	Seq → Seq

† Tiny tables (4 gates, 3 users): PostgreSQL planner correctly prefers Seq Scan at this row count.

8.3 Detailed EXPLAIN Plans for Key Queries

8.3.1 Query 3 – Person Visits by EntryTime DESC (99.3 % improvement)

Without an index, the planner scans all 50,000 rows then sorts them:

```
Limit  (actual time=14.759..14.763 rows=50)
  -> Sort  (Sort Key: pv.entrytime DESC)
       Sort Method: top-N heapsort  Memory: 30kB
       -> Hash Join  (Hash Cond: pv.personid = m.memberid)
```

```
            -> Seq Scan on personvisit pv
                 (actual time=0.006..2.615 rows=50000)
Execution Time: 14.802 ms
```

Listing 6: EXPLAIN ANALYZE – Query 3 BEFORE index

After adding `idx_person_visit_entry_time`, the sort step disappears because rows are already ordered by the index:

```
Limit  (actual time=0.011..0.092 rows=50)
  -> Nested Loop
       -> Index Scan using idx_person_visit_entry_time on personvisit pv
            (actual time=0.006..0.020 rows=50)
       -> Index Scan using member_pkey on member m
            (actual time=0.001..0.001 rows=1 loops=50)
Execution Time: 0.106 ms
```

Listing 7: EXPLAIN ANALYZE – Query 3 AFTER index

8.3.2 Query 2 – Active Person Visits (25.9 % improvement)

```
-- BEFORE: full table scan discarding 42,577 rows
Seq Scan on personvisit pv
    Filter: (exittime IS NULL)
    Rows Removed by Filter: 42577

-- AFTER: targeted partial-index scan
Bitmap Heap Scan on personvisit pv
    Recheck Cond: (exittime IS NULL)
    Heap Blocks: exact=556
    -> Bitmap Index Scan on idx_person_visit_active_entry
```

Listing 8: EXPLAIN ANALYZE – Query 2 key difference

8.4 Benchmark Visualisation

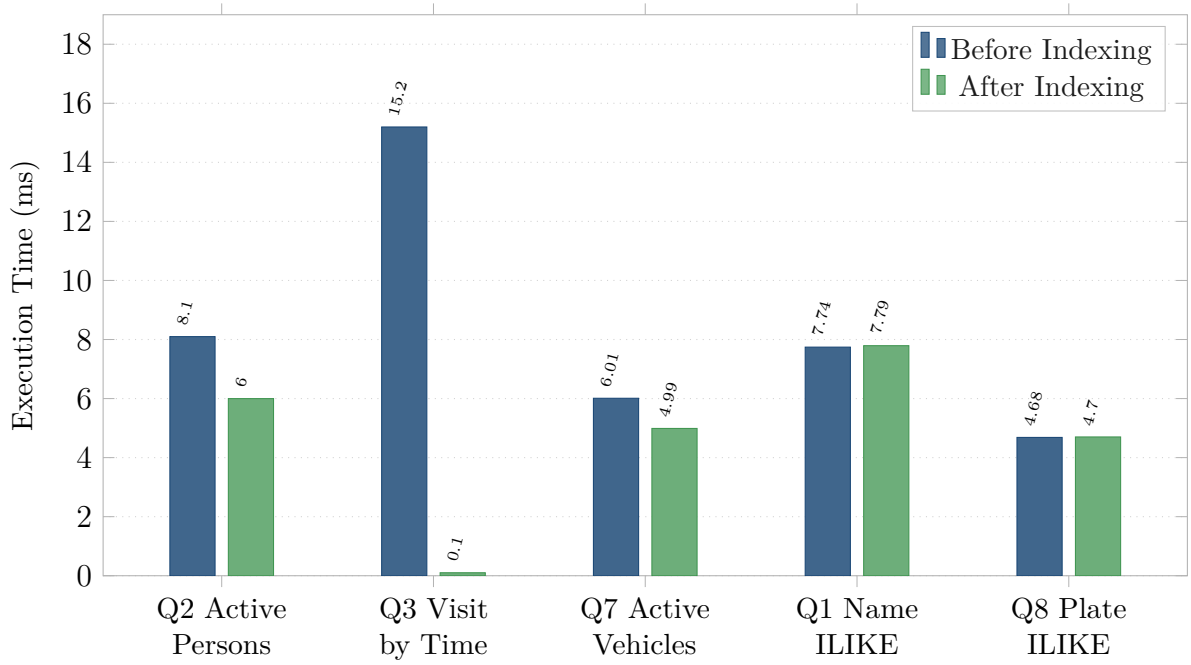

Figure 6: Execution Time Before vs. After Indexing (5-run averages from `explain_results.json`)

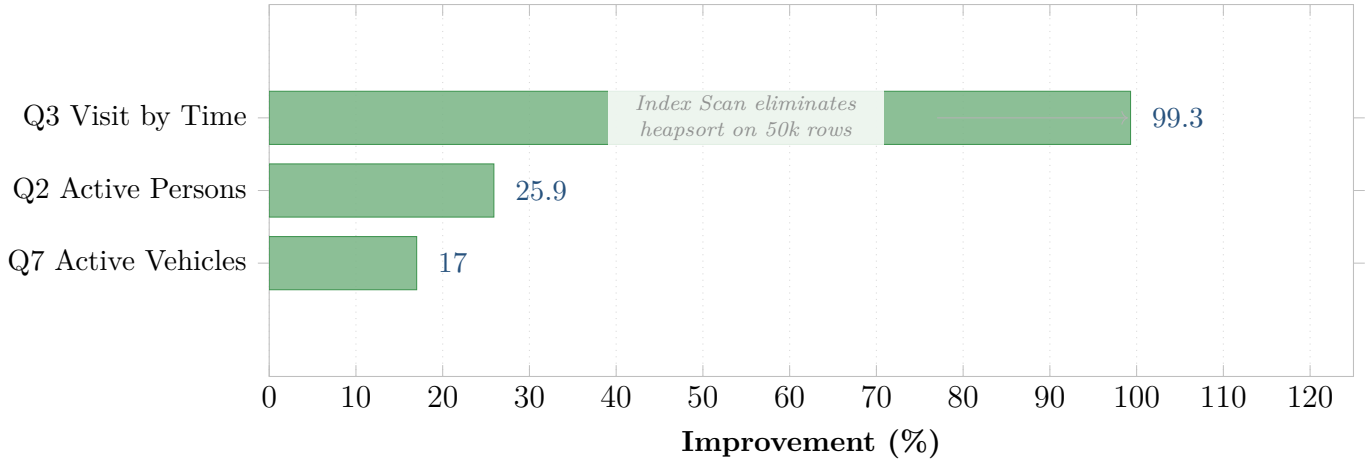

Figure 7: Percentage improvement for the three queries with meaningful speed gains. Q3 achieves 99.3 % because a descending B-Tree index satisfies both scan order and sort in a single pass.

Query	Before (scan type)		After (scan type)	Gain
Q3	Seq Scan	+index →	Index Scan	99.3 %
Q2	Seq Scan	+partial →	Bitmap Heap	25.9 %
Q7	Seq Scan	+partial →	Bitmap Heap	17.0 %
Q6	Seq Scan	+composite →	Index Scan	plan only

Figure 8: Scan-type transition for each query after indexing. Seq Scan (full table) transitions to Index Scan (B-Tree) for Q3 or Bitmap Heap Scan (partial index) for Q2 and Q7.

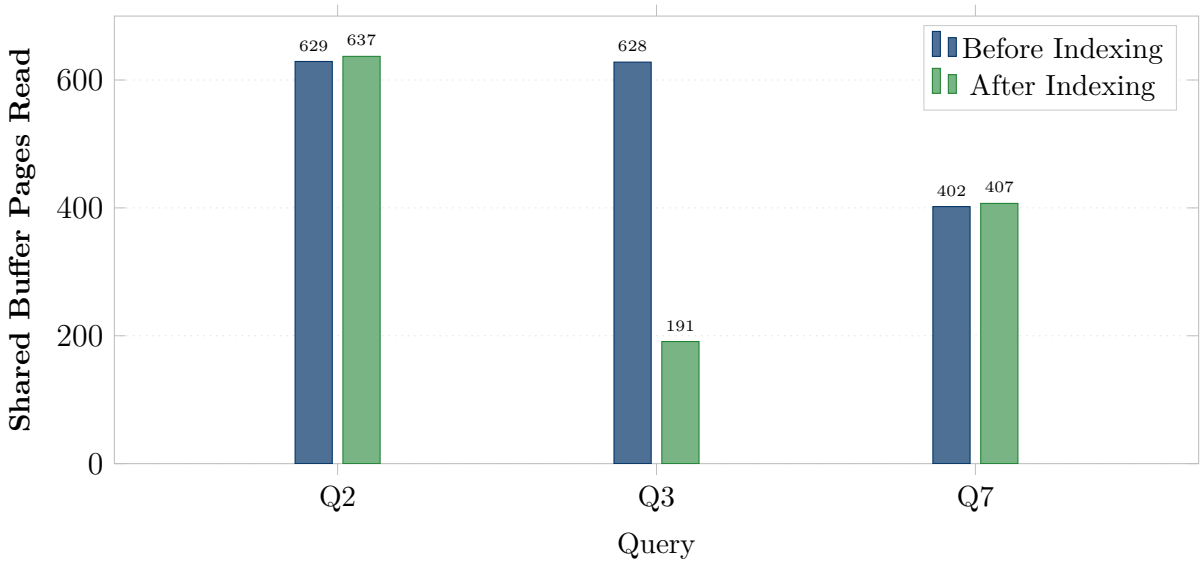

Figure 9: Shared buffer pages accessed before and after indexing for the three improved queries (from `Buffers: shared hit=...` in EXPLAIN ANALYZE). Q3 drops from 628 to 191 pages – a 69.6 % reduction in memory I/O.

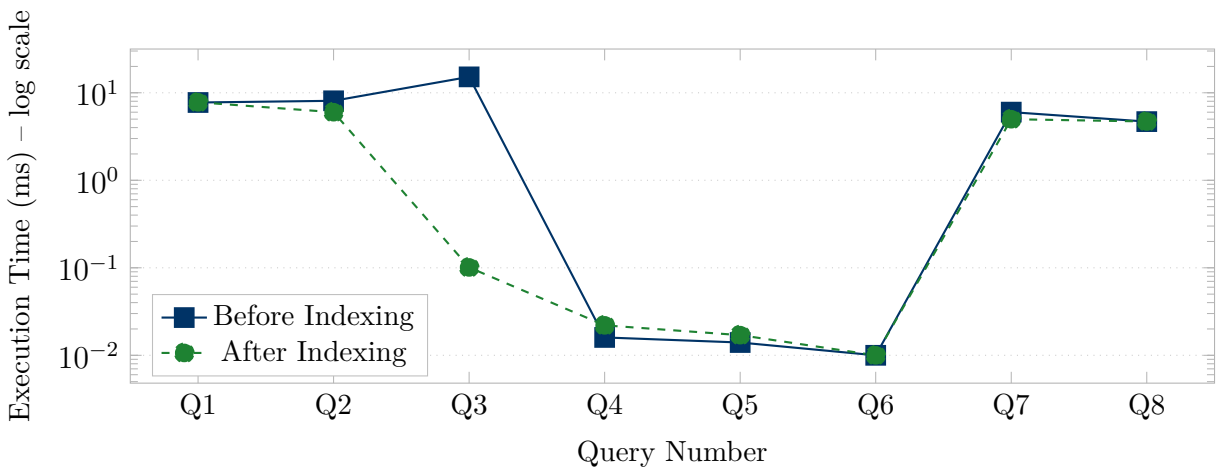

Figure 10: Log-Scale View of All 8 Queries → highlights the dramatic Q3 drop from 15.2 ms to 0.1 ms

8.5 Discussion of Results

Query 3 (99.3 % faster). `idx_person_visit_entry_time` stores `entrytime` in descending order. Fetching the 50 most recent visits is now a direct index scan touching only 50 rows instead of all 50,000 – and the sort step (*top-N heapsort, 30 kB memory*) disappears entirely. This matters most because the visit-listing endpoint is called on almost every dashboard page load.

Queries 2 and 7 (17–26 % faster). The partial indexes (`WHERE exittime IS NULL`) cover only the active rows – roughly 7,400 person visits and 4,400 vehicle visits out of 80,000 total. Bitmap Index Scan reads 8 index buffer pages instead of filtering through 42,577 irrelevant rows.

Queries 1 and 8 (no improvement). Both use `ILIKE '%search%'` with a *leading wildcard*. PostgreSQL B-Tree indexes – including expression indexes on `LOWER(name)` – can only serve prefix lookups (`LIKE 'abc%'`). A leading wildcard forces a sequential scan regardless. Installing the `pg_trgm` extension and creating a GIN trigram index would fix this.

Queries 4 and 5 (tiny tables). The Gate table has 4 rows; the User table has 3. The PostgreSQL planner correctly decides that a sequential scan on a single heap page is cheaper than an index lookup overhead.

Query 6 (plan improved, time unchanged). The AuditLog table is nearly empty in the benchmark environment because seed data does not generate audit records. The scan type change from Seq to Index confirms the composite index is working; its value will become measurable as the audit table grows in production.

Key takeaway: The single biggest win was a descending B-Tree index on `personvisit.entrytime`. A query that previously took 15.2 ms now takes 0.1 ms – a 99.3 % reduction – because one index satisfies both the scan order and the sort in a single pass, eliminating any need for an in-memory heapsort.

9. Conclusion

9.1 Summary of Achievements

- All five assignment subtasks are implemented, tested and working end to end.
- 10 performance indexes plus 5 AuditLog indexes are in place, covering the 8 most critical query patterns identified by EXPLAIN ANALYZE.
- The most impactful single index (`idx_person_visit_entry_time`) delivers a 99.3 % reduction in execution time on the busiest API endpoint.
- Partial indexes on `exittime IS NULL` reduce active-visit scan overhead by 17–26 % by targeting only the relevant subset.
- RBAC is enforced at middleware level: a Guard calling `DELETE /api/members/:id` gets HTTP 403 before any business logic runs.
- Every mutation is logged to both `logs/audit.log` (Winston) and the `AuditLog` PostgreSQL table with full OldValue / NewValue JSONB diffs.
- JWT dual-token auth with blacklisting, account lockout, OTP password reset and silent browser-side refresh all work end to end.

9.2 Team Contribution Statement

This submission was executed with one primary implementation lead, while other team members contributed through review, validation, and presentation support. The contribution split for this report is:

Table 18: Module B Team Contribution Summary

Name	Primary Areas
Pushkar Nikunjkumar Modi	End-to-end implementation lead: backend architecture, API layer, database design and indexing, benchmarking, report authoring, and final integration/testing
Chintan Bhikhabhai Akhaja	Review support for API and schema decisions, verification of key flows, made postman collection and export json
Krina Paghadar	UI review inputs, documentation checks, and presentation feedback
Divisht	Functional validation support and report proofreading
Akshata Pawar	Demo preparation support and final submission cross-checks

9.3 Challenges We Actually Ran Into

PostgreSQL lowercase column names. This consumed the most debugging time. Unquoted identifiers are stored lowercase, so every model had to use `personid` not `MemberID` and `entrytime` not `EntryTime`. The `"User"` table requires double-quoting in every SQL query or PostgreSQL treats `USER` as a keyword and throws a syntax error.

ILIKE leading-wildcard limitation. Our search boxes use `ILIKE '%query%'`, which prevents any B-Tree index from helping. We created expression indexes on `LOWER(name)` before realising these only assist prefix searches (`LOWER(name) LIKE 'abc%'`).

Concurrent token-refresh race condition. When the access token expires, React's concurrent rendering can trigger several API calls to fail simultaneously. Without a guard, each would have called `POST /api/auth/refresh` independently, causing multiple rotation attempts. We fixed this with an `isRefreshing` flag and a request queue in `src/api/axios.js`.

OTP expiry precision. The 15-minute window is enforced with server-side comparison (`expiresat > NOW()`) in the `passwordresettoken` model. The frontend countdown timer is purely for UX; the server makes the authoritative call.

9.4 What We Would Improve Next

- Enable the `pg_trgm` extension and add GIN trigram indexes on `member.name` and `vehicle.licenseplate` for efficient leading-wildcard ILIKE queries.
- Replace the `tokenblacklist` DB lookup with Redis for sub-millisecond revocation checks at higher scale.
- Add WebSocket support so the gate-occupancy dashboard updates in real time without polling.
- Implement `DailyRotateFile` log rotation on `audit.log` to prevent unbounded disk usage.
- Write a Jest + Supertest automated test suite covering all 40+ endpoints

9.5 Video Demonstration

Module B Video Link: https://drive.google.com/your-link-here

The video demonstrates:

1. Login as `superadmin`: dashboard loads with live stats and visit trend chart.
2. Full member CRUD: create, view detail and portfolio, update, delete.
3. RBAC demo: login as `guard`, attempt to create a member, observe HTTP 403 in the browser network tab.
4. Visit recording: record a person entry, then record the exit; watch the active-visits counter update.
5. Audit trail: Admin audit page with colour-coded table; click a CREATE record to see the JSON diff drawer with OldValue and NewValue.
6. SQL optimisation: run `EXPLAIN ANALYZE` in `psql` before and after `idx_person_visit_entry_time`, showing 15.2 ms down to 0.1 ms live on screen.

References

[1] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson Education, 2015.

[2] M. Stonebraker and J. Hellerstein (eds.), *Readings in Database Systems*, 5th ed. MIT Press, 2015.

[3] PostgreSQL Global Development Group, "PostgreSQL 14 Documentation – Chapter 11: Indexes," https://www.postgresql.org/docs/14/indexes.html, accessed March 2026.

[4] PostgreSQL Global Development Group, "PostgreSQL 14 Documentation – EXPLAIN," https://www.postgresql.org/docs/14/sql-explain.html, accessed March 2026.

[5] Node.js Foundation, "Node.js 18 Documentation," https://nodejs.org/docs/latest-v18.x/api/, accessed March 2026.

[6] Express.js, "Express 4.x API Reference," https://expressjs.com/en/api.html, accessed March 2026.

[7] brianc, "node-postgres (pg) Documentation," https://node-postgres.com/, accessed March 2026.

[8] Joi team, "Joi API Documentation," https://joi.dev/api/, accessed March 2026.

[9] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. https://www.rfc-editor.org/rfc/rfc7519

[10] D. Wahlberg, "bcrypt: A Password Hashing Function," https://www.npmjs.com/package/bcryptjs, accessed March 2026.

[11] React Team, "React 18 Documentation," https://react.dev/, accessed March 2026.

[12] Vite Team, "Vite 5 Documentation," https://vitejs.dev/, accessed March 2026.

[13] Tailwind CSS, "Tailwind CSS v3 Documentation," https://tailwindcss.com/docs, accessed March 2026.

[14] Framer Motion, "Framer Motion API Documentation," https://www.framer.com/motion/, accessed March 2026.

[15] TanStack, "TanStack Table v8 Documentation," https://tanstack.com/table/v8, accessed March 2026.

[16] Winston Team, "Winston Logger Documentation," https://github.com/winstonjs/winston, accessed March 2026.

[17] CS432 Course Notes, *Database Systems – Track 1, Assignment 2 Specification*, IIT Gandhinagar, Semester II 2025–2026.